

Image based self localization in interplanetary space

Frederic Heil

Ravensburg-Weingarten University of Applied Sciences

October 22, 2025

Abstract

This paper presents image based solutions for a hypothetical spacecraft to localize itself within the solar system. The spacecraft possesses a gyroscope and camera systems for position determination. The first step for full localization is to determine and turn the spacecraft in the direction of the sun. The direction is determined using a wide field camera and a surveying procedure. A polynomial corrective model is used to rectify radial lens distortion. Next up, the distance to the sun is determined by making use of a simple geometrical model. The camera properties for this approach are partly modeled after a real spacecraft camera. Performance results are put into context of actual spaceflight. Afterwards attitude is determined against the background stars. To accomplishing this, stars need to be matched to a star catalog. Subsequently, approaches for optimizing the matching procedure by modifying camera parameters and attitude determination procedure are evaluated.

Implemented solutions located here: <https://github.com/stonkchonk/interplanetary-localization>

1 Introduction

There are currently various approaches for spacecraft to be localized or self localize in space. Some of them require external input like signal timing, where distance is determined by measuring the time a signal takes to reach the probe [1].

Attitude determination can be done with an onboard device called star tracker [2].

This paper explores a purely image based approach such that a spacecraft could autonomously localize itself using onboard software and hardware, specifically camera equipment, without external input.

For developing, evaluating and optimizing we will use the simulation software Space Engine. Space Engine is a realistic simulation that incorporates a virtual universe including the solar system and nearby, real stars among other things [3]. On top of simulating a universe, this software allows for running custom scripts and creating virtual cameras. It is possible to precisely control parameters, like field of view, heading angles, exposure time and position of a virtual camera.

All camera settings that will be defined in the upcoming sections are similar to those defined in Space Engine during development.

1.1 Scenario and constraints

To understand where the following sections arise from, a scenario has to be exemplified which entails some problems. Let there be a fictional spacecraft in interplanetary

space within the solar system. The spacecraft does not possess any information on where it is located relative to the sun, has no means of communicating with a ground station and has to perform a maneuver based on its location autonomously. Additionally, the spacecraft should be randomly aligned against the background stars and it does not know where the sun is located relative to itself. With all the information the spacecraft does not possess, there is some information and some tools it does possess in order to determine the location of the sun relative to itself, determine the distance from the sun, determine its attitude relative to the background stars and therefore obtain its actual position within the solar system.

First of all, the spacecraft should contain a gyroscope so that it can perform accurate turns whilst knowing how much it turned relative to its previous orientation. This does not mean that it knows its attitude in relation to the background stars, rather that it just turned by e.g. 90° along one of its axes. Secondly, the spacecraft should contain camera systems so that it can obtain information on sun location, distance and attitude to the background stars. Furthermore, it stores the information of a star catalog so that it knows where stars are located relative to each other.

With this information and these tools it should be possible to determine its location using means of computer vision.

1.2 Coordinate system

There are several celestial coordinate systems to choose from. Some might be more intuitive for describing the position of our spacecraft. However, as coordinates can be transformed into other coordinates systems it does not matter. We are simply choosing the one which best fits our needs.

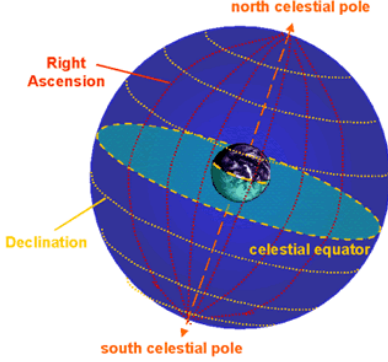


Figure 1: The equatorial coordinate system being aligned to Earth's equator.[4]

A key aspect of attitude determination relative to the background stars is to know where stars are located relative to each other. One such coordinate system, which is used to describe positions of stars is the equatorial coordinate system. The coordinate system in question is a spherical coordinate system which uses **Right Ascension** and **Declination** to pinpoint objects in the sky of planet Earth. Distance is not taken into account. [4] This system is not yet sufficient to describe positions within the solar system as it does not take distance into account and is centered around Earth.

Our coordinate system of choice places the sun at the center, maintains the celestial equator plane and adds a distance component.

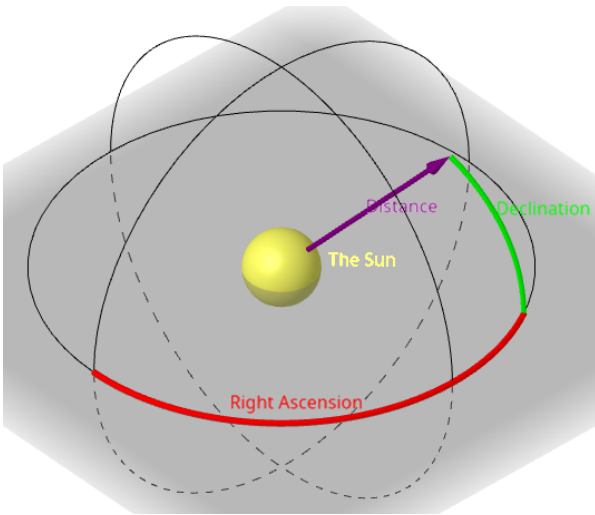


Figure 2: Any three dimensional position can be described by right ascension, declination and distance.

Using right ascension, declination and distance, it is possible to determine how far the spacecraft is from the sun and where it is located relative to the background stars, which satisfies a description of any position in three dimensional space.

2 Localization of the sun

The scenario description states that initially, our spacecraft should be randomly aligned against the background stars and does not know where the sun is located relative to itself. And this will be the first goal: Locating the sun, specifically pointing a camera precisely at the sun. Solving this problem is crucial for solving the follow up tasks, namely determining distance in section 3 and determining attitude in section 4.

2.1 Sun detection procedure

The general approach is to assume that the sun is always the brightest object in the sky. To detect it, the whole surrounding sky needs to be surveyed. A simple surveying procedure is preferable, therefore we aim to take only very few pictures to detect and center the sun.

To capture the entirety of the sky, six images need to be taken that resemble something like a skybox. A skybox in the context of gaming consists of six square images facing up, down, left, right, towards the front and towards the back [5] as stated by the Valve documentation. Using a camera with a minimum field of view of 90° by 90° we can define a virtual camera and a procedure on how to turn the spacecraft and obtain the six images.

Camera Property	Value
Image frame [pixels]	1000 x 1000
Field of view [°]	92 x 92
Exposure time [s]	$\frac{1}{8192}$

Table 1: Sun detection camera properties.

Our defined camera actually has a slightly larger field of view in case of the sun being located right on an edge of an image, which could be a rare edge case. This is being done to make sure that there is always enough of the sun in an image to be detectable. Furthermore, the exposure time is quite short to assure that only the sun gets detected and not any other background stars.

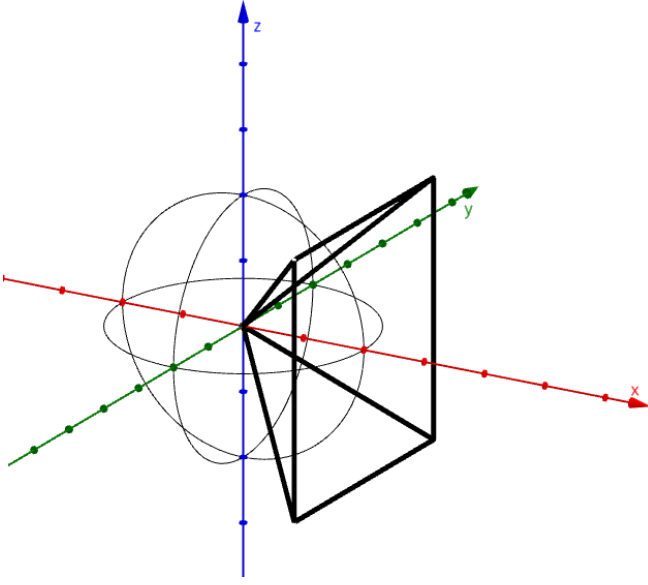


Figure 3: Initial camera frame heading of the sun detection procedure and spacecraft rotation axes.

The above Figure 3 shows the initial heading of the sun detection camera pointing along the x-axis, and how y-axis and z-axis are defined in this context. The imaging procedure is to be conducted as follows:

1. Take the front facing image.
2. Rotate counterclockwise around the y-axis by 90° .
3. Take the upward facing image.
4. Rotate counterclockwise around the y-axis by 90° .
5. Take the backward facing image.
6. Rotate counterclockwise around the y-axis by 90° .
7. Take the downward facing image.
8. Rotate counterclockwise around the y-axis by 90° .
9. Rotate counterclockwise around the z-axis by 90° .
10. Take the left facing image.
11. Rotate clockwise around the z-axis by 180° .
12. Take the right facing image.
13. Rotate counterclockwise around the z-axis by 90° .
The spacecraft is now once again in its initial position.

The procedure yields six images resembling a skybox as shown in Figure 4.

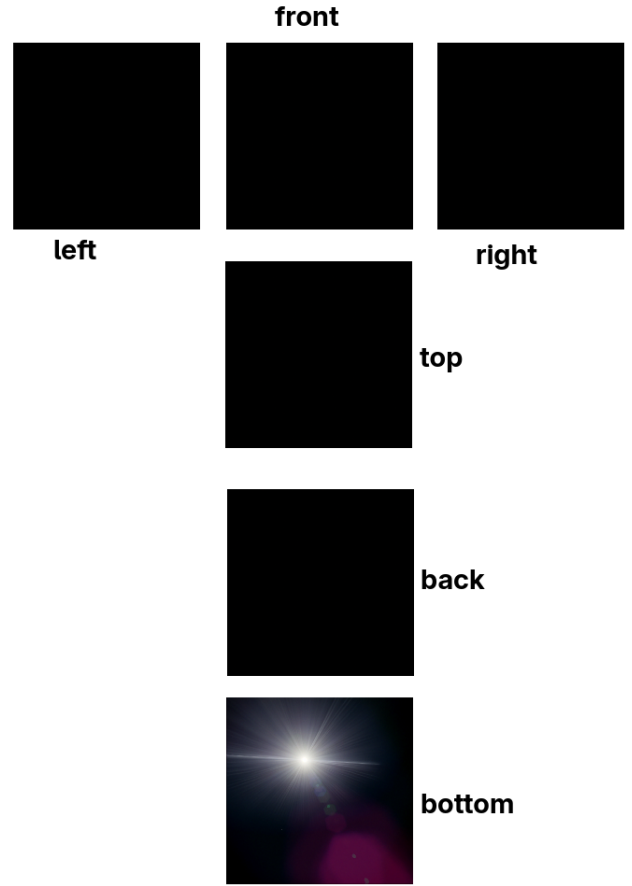


Figure 4: Six resulting images of the sun detection procedure yielding the general direction of the sun.

From these six images we can already determine the general direction of the sun relative to our spacecraft. In this case the sun is located below relative to the spacecraft. Every black picture can be excluded and usually there is only one bright picture containing the sun which can be chosen as the general direction. In some cases there might be multiple frames containing parts of the sun caused by overlap due to the larger than 90° by 90° field of view. Another reason for the sun being in multiple frames might be the spacecraft being located close to the center of the solar system, thus the sun visually occupying more of the surrounding area.

By converting the images to grayscale and applying some simple thresholding we can get rid of diffraction spikes and other lens effects. This also yields a mask image whereas the mask part (white pixels in otherwise black image) represents the actual solar disk.

The image/mask containing most white pixels equates to the general direction of the sun, hence the direction to point the camera at.

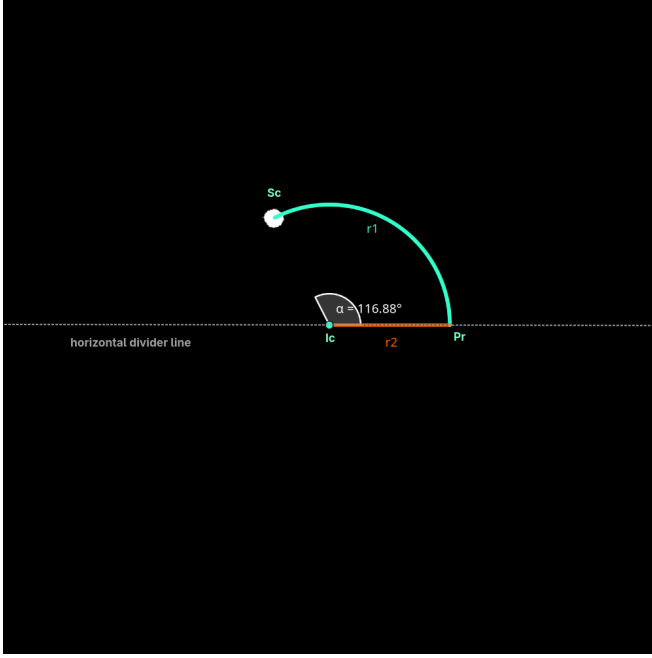


Figure 5: Mask image of the sun containing two rotation operations for centering the sun within the frame.

To make the camera face the sun directly two rotation operations r_1 and r_2 have to be performed which is illustrated in Figure 5.

The goal of r_1 is to place the sun on an imagined horizontal divider line on the right side of the image center. For r_1 the camera has to be rotated anticlockwise around the axis which is facing the previously determined general direction by angle α . The value of α can be defined as the angle between the horizontal divider line and the imagined vector between the image center I_c and sun center S_c points.

Pixel wise the image center can be defined as $I_c = \begin{pmatrix} 499.5 \\ 499.5 \end{pmatrix}$ because the image's width and height span 1000 pixels whereas the pixels are numbered from 0 to 999. The sun center S_c can be defined as the mean position value of all white pixels within the mask image which is outlaid in the following equation 1:

$$S_c = \frac{1}{n} \sum_{i=1}^n \vec{p}_i \quad (1)$$

Hereby n is the number of all white pixels and $\vec{p}_i$ equates to the position vector of every pixel i .

The goal of r_2 is to move the sun from a point right to the image center P_r placed on the horizontal divider line to the image center I_c itself. For this the camera has to be rotated along the axis perpendicular to the horizontal line by an angle β . If our frame were to be perfectly rectilinear, the rotation angle could be calculated relatively easily with the following equation 2:

$$\beta = FOV * \frac{l(r_2)}{w} = 92^\circ * \frac{|P_r - I_c|_{px}}{1000px} \quad (2)$$

Hereby we determine the offset of P_r relative to I_c and from this the pixel wise fraction this offset occupies relative to the width w of the image. This value is then multiplied with the field of view value which yields the resulting degrees value for β .

This approach would work without issues for any rectilinear image. However, as we using a wide angle camera which captures images of 92° by 92° this is not the case and we have to account for radial lens distortion.

2.2 Correction of lens distortion

As we are dealing with a radially distorted and not a perfectly rectilinear image, we have to define a method for correcting that distortion. Even more specifically we are trying to determine the actual value of β for the second rotation r_2 as explained in the previous section, so a full image correction is not necessary, just a correction of that distortion.

The paper by Prescott and McLean presents an approach for full correction of radial distortion, which we can adapt for our purposes. Before stating the corrective equations, we have to define some of its variables beforehand. The coordinate (x, y) refers to any point in the rectilinear image, (x', y') refers to the corresponding point in a distorted image, $(\hat{x}, \hat{y})$ refers to the corrected coordinate and (c_x, c_y) represents the center coordinate of the image. The following equations 3 state how $\hat{x}$ and $\hat{y}$ are deduced:

$$\begin{aligned} \hat{x} &= x' + \bar{x}(K_1 r^2 + K_2 r^4 + K_3 r^6 + \dots) \\ \hat{y} &= y' + \bar{y}(K_1 r^2 + K_2 r^4 + K_3 r^6 + \dots) \end{aligned} \quad (3)$$

Hereby $\bar{x}$ and $\bar{y}$ represent the offset of x' and y' from their corresponding center values c_x and c_y as follows:

$$\begin{aligned} \bar{x} &= x' - c_x \\ \bar{y} &= y' - c_y \end{aligned} \quad (4)$$

Furthermore, r is defined as a radius or absolute distance of a point (x', y') from the image center (c_x, c_y) as follows:

$$r = \sqrt{(x' - c_x)^2 + (y' - c_y)^2} \quad (5)$$

$K_1, K_2, K_3, \dots$ are distortion parameters. In their paper Prescott and McLean only focus on K_1 and K_2 which yields a less complex corrective function. [6]

For our approach we are only interested in an offset which depends on the radius or absolute distance of a point from the image center. To do that, we are going to modify the approach of Prescott and McLean to formulate an offset function $d(r)$ which does not need any image coordinates, just the value of r .

As specific coordinates are not relevant, we are only going to look at the first equation of 3. The first step is to subtract x' from both sides which yields the next equation:

$$\hat{x} - x' = \bar{x}(K_1 r^2 + K_2 r^4 + K_3 r^6 + \dots) \quad (6)$$

Up next, looking at terms 4 and 5 it becomes apparent that $\bar{x} = r$ if $y' = c_y$ and $x' \geq c_x$. As we want to eliminate specific coordinates from our equation and make it dependent on the radius, we are going to insert $\bar{x} = r$ into equation 6 which yields:

$$\hat{x} - x' = r(K_1r^2 + K_2r^4 + K_3r^6 + \dots) \quad (7)$$

In general, the offset we want to determine can be defined as the difference between an undistorted value and the corresponding distorted value as $d = x - x'$ or as the difference between the predicted value and the corresponding distorted value as $d = \hat{x} - x'$. Specifically the second term is already part of equation 7 on the left side. To make our corrective model more accurate, we want to reformulate that term a little.

Ideally the predicted values match the undistorted value as in $\hat{x} = x$, but this is probably not the case, as there will always be some error e as in $\hat{x} = x + e$ where e can take a positive or negative value. However, this constant value would not work for the center of the image where $r = 0$ because at this point the predicted value must exactly match the undistorted value as in $\hat{x} = x$ and e may be nonzero.

To solve this issue, we can make e dependent on r by formulating it as a linear function $e = ar$. We can then insert these terms into equation 7 and move the error part to the right side:

$$\begin{aligned} \hat{x} - x' &= r(K_1r^2 + K_2r^4 + K_3r^6 + \dots) \Leftrightarrow \\ x + e - x' &= r(K_1r^2 + K_2r^4 + K_3r^6 + \dots) \Leftrightarrow \\ x + ar - x' &= r(K_1r^2 + K_2r^4 + K_3r^6 + \dots) \quad | - ar \Leftrightarrow \\ x - x' &= r(K_1r^2 + K_2r^4 + K_3r^6 + a + \dots) \end{aligned}$$

It is important to note that this measure would probably not predict x perfectly correct as in $\hat{x} = x$. To make the equation formally correct, we are going to replace x by $\hat{\hat{x}}$ where $\hat{\hat{x}}$ corresponds to an improved predicted value such that $|x - \hat{\hat{x}}| \leq |x - \hat{x}|$. This yields

$$\hat{\hat{x}} - x' = r(K_1r^2 + K_2r^4 + K_3r^6 + a + \dots)$$

where we can replace the right side with our offset function term $d(r)$. Furthermore, the error value a is also a coefficient, which will be denominated as K_0 . Summarized, this results in a polynomial function which does not depend on any coordinates but just on the absolute distance from the image center r and coefficients K_i :

$$d(r) = K_0r + K_1r^3 + K_2r^5 + K_3r^7 + \dots \quad (8)$$

In order for this to work properly as a corrective function, we have to define a procedure on how to determine the coefficients.

One approach, described by M. Schneider, is to define and minimize the error on an objective function for a nonlinear model so that it yields the coefficients we are looking for. The objective function can be described as

follows:

$$J(w) = \frac{1}{m} \sum_{i=1}^m l(h(x_i), y_i) \quad (9)$$

Hereby m is defined as the number of data points, l as the loss function, $h(x)$ as the hypothesis function and y as the values to fit our model to. To represent a nonlinear context, $h(x)$ can be defined as the dot product between a weights vector w and a feature function $\phi(x)$:

$$h(x) = \langle w, \phi(x) \rangle \quad (10)$$

[7]

This can now be applied to our previously defined corrective function. The coefficients K_i can be put into the weights vector w , while the polynomial model can be defined as a feature vector $\phi(r)$ which yields our hypothesis function. For our purposes, as it seems to produce sufficient results, we are going to limit the resulting polynomial to four components. This yields the hypothesis function $h(r)$:

$$h(r) = \langle (K_0, K_1, K_2, K_3)^T, (r, r^3, r^5, r^7)^T \rangle \quad (11)$$

Up next, we can define the value vector y . Hereby, its elements represent the difference in terms of r between a supposed rectilinear point and its corresponding measured point. This step can be automated using our camera defined in Space Engine. We can determine a supposed rectilinear point or distance from the image center (c_x, c_y) , turn it accordingly and measure the corresponding observed distance.

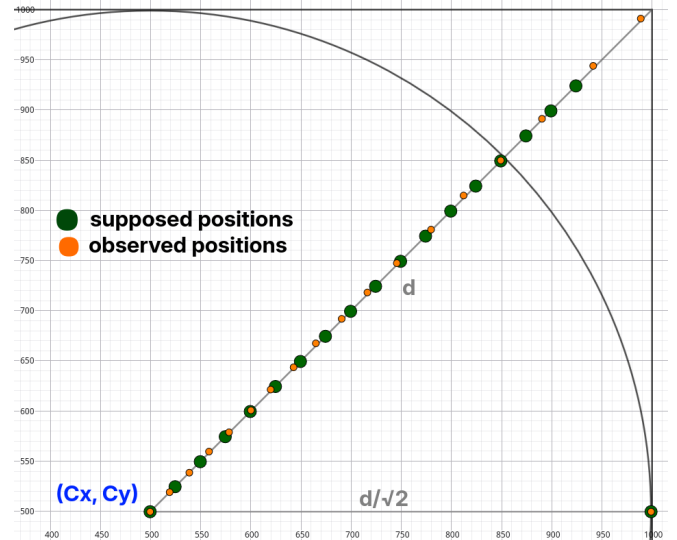


Figure 6: Turning the camera by a set amount and comparing measured positions with supposed rectilinear positions.

Figure 6 shows how this can be executed. At first, the camera is being pointed directly towards the sun so that it is placed in the center of the image. After that it is turned by a set angle diagonally so that the sun appears

to move towards the corner of the frame. Once this is done, the new position of the sun (represented by orange dots) is measured and the difference to its corresponding supposed distance (represented by green dots) is stored as a value for the y vector.

This process has to be repeated several times for multiple angles so that the result can be modeled as accurate as possible. Moreover, any distance value is normed in terms of d so that it lies between 0 and 1. This makes the minimization process much more stable and guarantees for a usable result. Hereby d is half of the image diagonal spanning over $92^\circ/\sqrt{2} = 65.05^\circ$.

Now that all parameters are known, we can minimize the objective function using squared loss for l by performing stochastic gradient descent. Once the loss has been minimized, we can extract the weights and use them as coefficients for the corrective function 8 which results in:

$$d(r) = 0.27727r - 0.78599r^3 + 0.51587r^5 - 0.14964r^7 \quad (12)$$

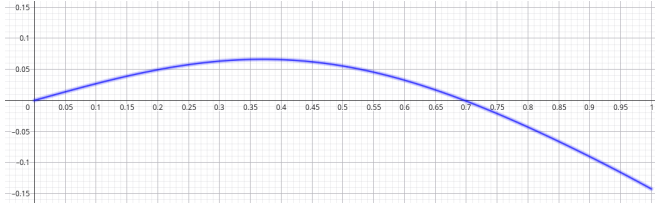


Figure 7: Plot of $d(r)$ displaying the offset in terms of d for any given input value between 0 and 1.

At last, this corrective function can be used to determine the actual turning angle β of equation 2 to perform an accurate rotation r_2 using a distorted image. Equation 2 has to be slightly modified as follows:

$$\begin{aligned} d_w &= \frac{l(r_2) * \sqrt{2}}{w} \\ \beta &= FOV * \frac{l(r_2) + d(d_w) * \frac{w}{\sqrt{2}}}{w} \end{aligned} \quad (13)$$

In this modified equation we first determine the length of the second rotation $l(r_2)$, a pixel value, in terms of d as d_w . To determine the corrective value, d_w needs to be inserted into $d(r)$. As shown in Figure 6 the function's input and output value are represented in terms of the diagonal d . To account for the output type, $d(d_w)$ gets multiplied by $\frac{w}{\sqrt{2}}$ to transform it to a pixel value once again.

With these results it is possible to pinpoint the location of the sun relative to the spacecraft and turn it accordingly. Pointing a camera towards the sun will be crucial for the upcoming tasks.

3 Determining distance to the sun

As shown in Figure 2, we are using a spherical coordinate system to describe the location of our spacecraft within the solar system. An important value of that description is the distance to the sun, the determination of which is the main topic of this section.

The approach to determine distance is quite simple. The sun is a spherical object with a somewhat fixed radius. It is important to note that the solar radius is indeed increasing as time passes but in such a slow manner (millions of years) [8] that we can assume this value to be a known constant.

3.1 Procedure

With that information we can define a geometrical model and a procedure on how the distance can be determined using images.

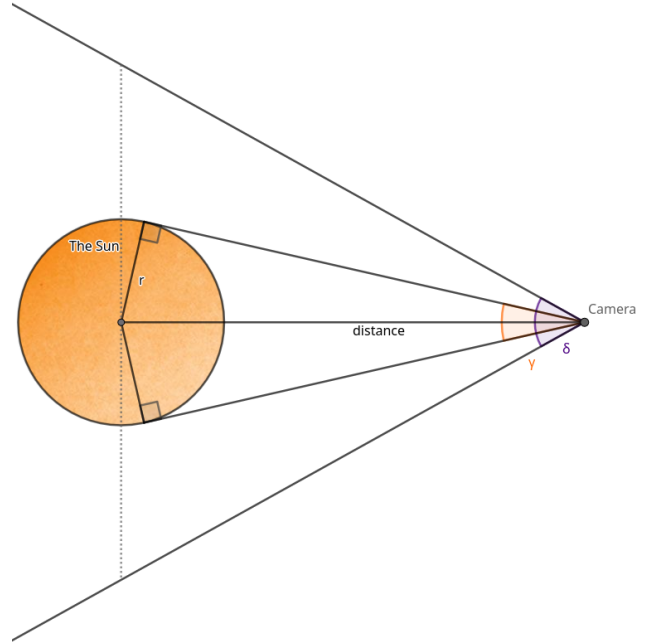


Figure 8: Geometrical model showing the relation between distance and apparent diameter of the sun.

As shown in Figure 8, three values are crucial for distance determination, namely the camera's field of view δ , the apparent solar diameter γ within an image and the solar radius r_s . The relation between the apparent diameter γ and solar distance d_s can be expressed as follows:

$$d_s = \frac{r_s}{\sin\left(\frac{1}{2} * \gamma\right)} \quad (14)$$

The apparent diameter γ can be determined from an image if the image's field of view value δ is known and can be calculated as follows:

$$\gamma = \delta * \frac{s}{w} \quad (15)$$

Hereby s corresponds to the solar diameter in pixels and w to the image's width or height in pixels corresponding to the field of view δ .

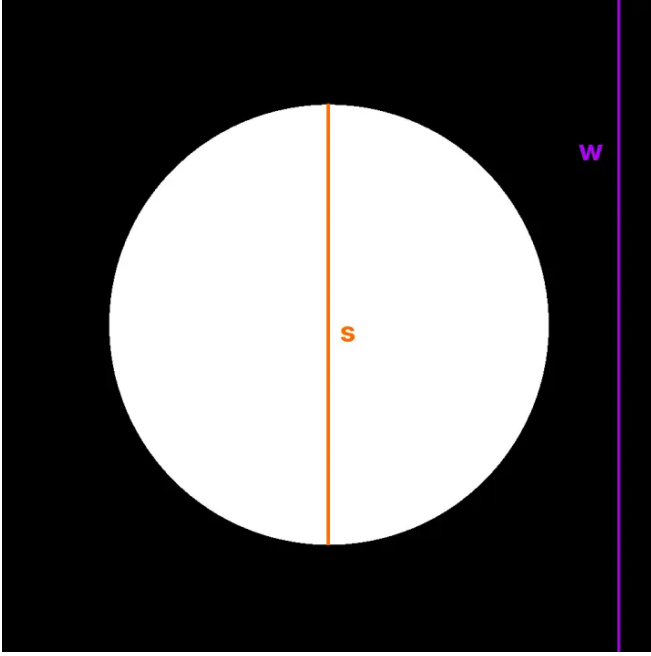


Figure 9: Mask image of the sun from which the apparent diameter γ can be determined.

To extract s precisely, the corresponding image should be a mask image containing only black and white pixels where white pixels are part of the solar disk and black pixels are not. Such a mask image can be obtained by converting the original image to grayscale and applying some simple thresholding, which yields an image similar to Figure 9.

Now that we have established the general principle for distance determination, we can derive a procedure and concrete camera parameters. As the determination of the apparent diameter γ in equation 15 relies on the division of whole-number values, we want the solar pixel diameter s to be relatively large compared to the image width w . That is because the actual apparent diameter does not change in an incremental or decremental way as it is with the mask pixels, rather continuously. If s is small, like only a few pixels, the actual angular diameter may be slightly smaller or larger, which results in a more distinctive error. Also s should not be larger than w because we can only measure the sun's apparent diameter if it fits within the image.

With these constraints we can choose the camera's field of view angle. In particular, it should not be just one angle, rather it should be possible for the camera to use multiple field of view angles, so that it can zoom in and out. As the image frame remains constant with 1000 by 1000 pixels, we have to choose the minimum and maximum field of view angle. Specifically the minimum value is of importance because it corresponds to the camera

being zoomed in to its maximum.

As we want these settings to be somewhat realistic, they should be based off of an actual spacecraft camera, more specifically the Long-Range Reconnaissance Imager, in short LORRI, of the New Horizons spacecraft.

The LORRI camera is a device for taking high resolution images with a field of view spanning 0.29° by 0.29° and an image frame of 1024 by 1024 pixels. These settings yield a per pixel resolution of $4.94 \mu\text{rad}$. [9]

For our purposes, we are going to use these values as guiding values and give our camera a slightly better per pixel resolution. Our distance camera's minimum field of view should span 0.20° by 0.20° with an image frame of 1000 by 1000 pixels which yields a per pixel resolution of $3.49 \mu\text{rad}$. Next up, we define the maximum field of view value as 12.8° by 12.8° , which yields a per pixel resolution of $223.4 \mu\text{rad}$. We choose these boundaries specifically because it allows for observing the sun over a wider range, while also avoiding any lens distortion effects. Furthermore, these values are of importance for the imaging procedure.

Camera Property	Value
Image frame [pixels]	1000 x 1000
Field of view $^\circ$	$0.2 \times 0.2 - 12.8 \times 12.8$
Exposure time [s]	$\frac{1}{8192}$

Table 2: Sun distance estimation camera properties.

As stated previously, we would like the pixel diameter s of the sun to be somewhat large compared to the image width w , but also not too large, so that the sun always fits within the frame. To assure this, an imaging procedure can be defined with the following flow chart:

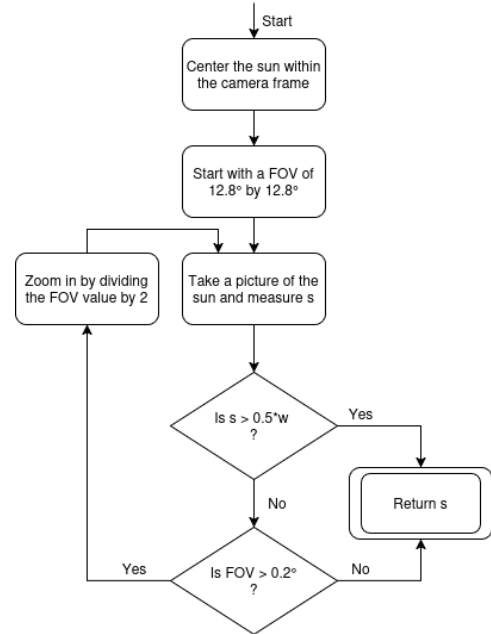


Figure 10: Simple imaging procedure to determine a value for the solar pixel diameter s .

As illustrated above in Figure 10, the camera is being pointed towards the sun with an initial field of view of 12.8° by 12.8° . Then it proceeds by taking pictures of the sun and halving its field of view until it either cannot reduce its field of view anymore or the sun occupies more than half of the image's width in terms of pixels. Once that state is reached, the sun's apparent diameter can be determined using the resulting pixel diameter s and the corresponding field of view value δ of the image, using equation 15.

This process is also the reason why the seemingly arbitrary value of 12.8° has been chosen as the upper bound for the camera's field of view, as it can be divided by two exactly six times.

3.2 Accuracy and limitations

Now that a mathematical model, procedure and camera properties have been defined, we can evaluate the accuracy and limits of this system. However, the results of this stem mostly from camera properties, so that more beneficial properties might yield better results. Furthermore, Space Engine, the simulation software responsible for rendering test images, also drastically influences any result. That is why concrete results have to be taken with a grain of salt, although the procedures used to determine accuracy are genuine.

As previously stated with equation 15 and shown in Figure 9, the solar diameter s within an image equates to the number of pixels the sun occupies in terms of width. As pixels are counted as whole-number values and not fractional, this in itself limits the accuracy of the distance calculation which becomes especially apparent at larger distances as error builds up.

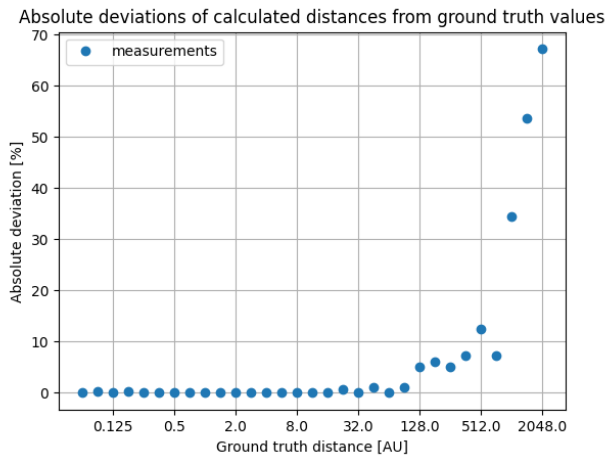


Figure 11: Absolute deviation of calculated distance from ground truth distance over a large range of distances.

Illustrated above in Figure 11 are the absolute deviations of calculated distances from their corresponding ground truth distance values. This dataset has been created by

placing the virtual camera at certain distances in Space Engine and running the distance determination procedure. As the sun becomes more distant and less pixels are being used for the solar diameter s to determine the apparent diameter, γ becomes less accurate. At some point above a distance of 512 astronomical units, the error rises drastically which marks the upper limit up to which the distance determination can be used to determine approximate results. Below a distance of 128 astronomical units, the distance is usually quite accurate.

To put things into perspective, one astronomical unit equals the mean distance between planet Earth and the sun, whereas the dwarf planet Pluto is located around 40 astronomical units from the sun [10].

Next up, we are going to determine the upper limit of distance estimation abilities precisely by comparing concretely calculated values with their ground truth distance, as illustrated below in Figure 12.

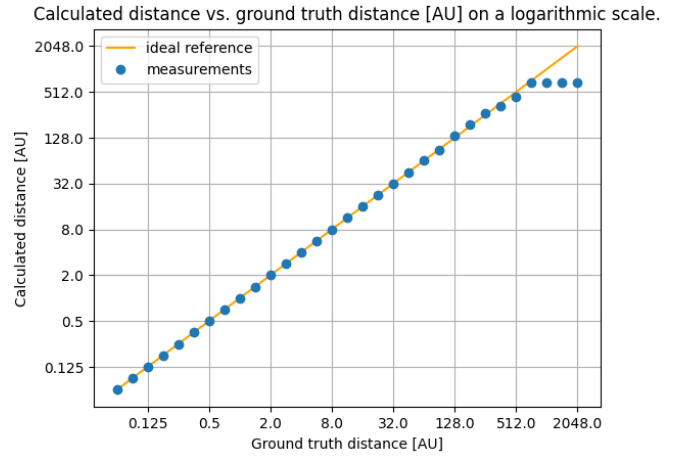


Figure 12: Absolute deviation of calculated distance from ground truth distance over a large range of distances.

At some ground truth distance above 512 astronomical units, the calculated distance stays constant, which is also the reason why the absolute deviation increases drastically after this point.

This phenomenon may be explained by some Space Engine rendering process, as we can precisely calculate the upper value by assuming a solar pixel diameter s of four pixels and a camera field of view δ of 0.2° . Inserting these two values into equation 15 yields the apparent diameter γ , which when inserted into equation 14 yields a calculated distance $d_s = 666.12 AU \approx 10^{11} km$, which can be considered as the upper limit of any practical predictability within the simulation software.

The lower limit, meaning the closest point to the sun, where distance can be measured reliably, can be derived from the maximum field of view of the camera and the fact, that distance can only be measured if the solar disk fits within an image. The maximum field of view δ can be set to 12.8° , which is also the maximum apparent di-

ameter γ to be measurable. Plotting this value into equation 14 yields a calculated distance of $d_s = 0.0417AU \approx 6.24 * 10^6 km$.

To make sense of these limits and determine how useful this camera system would be in real life, they can be compared to extremely close and far distances which actual probes have reached within the solar system.

One example for very close solar encounters is the Parker Solar Probe. This probe has reached a minimum distance of $6.2 * 10^6 km$ from the surface of the sun [11]. As distances in our model are measured from the center of the sun and not from the surface, the lower limit of reasonable measurements is slightly closer than the probe's minimum distance.

As for the upper limit, no probe in existence has even come close to reaching that distance from the sun. However, the currently (as of October 2025) furthest probe from the sun is the Voyager 1 spacecraft with a distance of $168.6AU$ [12]. This value is already in a range where our camera system starts getting inaccurate in terms of distance predictions.

4 Determining attitude to the background stars

Determining the attitude to the background stars is the next step for full self localization in interplanetary space after the distance to the sun has been determined. Within our established spherical coordinate system, attitude refers to right ascension and declination, as shown in Figure 2. Our general approach to determine attitude is to take images of the stars, match observed stars within these images with stars in a catalog, determine a view vector and finally determine the attitude from that view vector. Furthermore, we are going to take a look at optimization approaches regarding the star matching algorithm, as well as the attitude determination procedure.

The star catalog of our choice is the BSC5P - Bright Star Catalog, which contains stars brighter than apparent magnitude 6.5, as well as their position coordinates in right ascension and declination among other things [13].

4.1 Determining the view vector

Determining a view vector from an image containing stars works under the assumption that the stars within the image have been matched and identified. Chronologically this step has to be conducted after stars have been matched, however it makes sense to define it beforehand as it poses a constraint on the matching algorithm. To determine a view vector, three known stars need to be observable, so that the corresponding vector can be calculated. Herein lies the constraint, namely that at least three stars need to be matched to conduct the following step.

Before elaborating the method of calculation, we have to transform all star coordinates into unit vectors, which is

necessary to perform the following tasks. Star positions in relation to the equatorial coordinate system are given as two angles, right ascension α and declination ρ . The paper by Ying, Fei and Zheng, which also contains the matching algorithm, relays an equation for converting these angles to a unit vector v :

$$v = \begin{bmatrix} \cos(\alpha) \cos(\rho) \\ \sin(\alpha) \cos(\rho) \\ \sin(\rho) \end{bmatrix} \quad (16)$$

The advantage of unit vectors for referring to positions over two angles is the fact that it is easily possible to calculate the angular separation or cosine separation between them as follows:

$$v_i \cdot v_j = |v_i| * |v_j| * \cos(\theta) = \cos(\theta) \Leftrightarrow \arccos(v_i \cdot v_j) = \theta \quad (17)$$

[14]

Hereby, θ equates to the angular separation and the cosine of θ to the cosine separation. Also because we are using unit vectors with unit length 1, it is safe to assume that $|v_i| * |v_j| * \cos(\theta) = \cos(\theta)$.

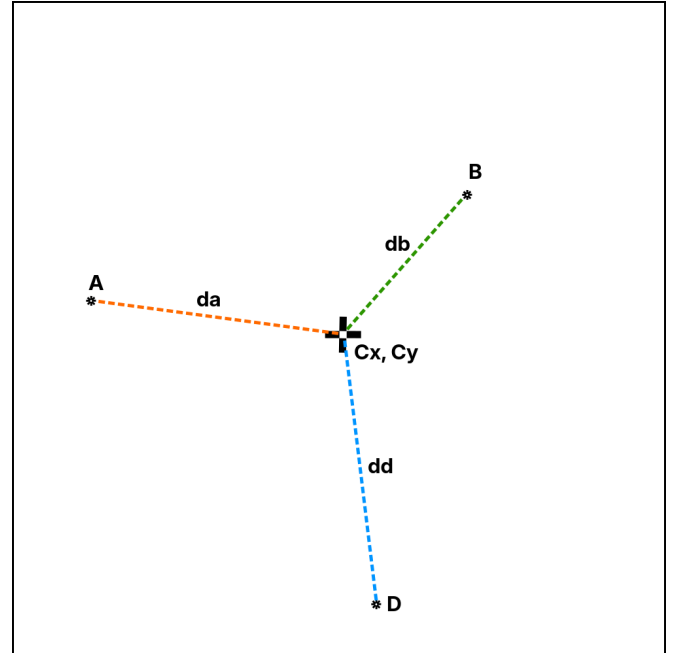


Figure 13: Triangulation of a view vector within a rectilinear image from three pixel positions of known stars.

As stated before and as illustrated in Figure 13, three known stars must be visible within an image frame to determine a view vector. Hereby a view vector corresponds to a vector pointing in the direction of the image's center. These three stars A , B and D are located at certain pixel positions, which are separated by the corresponding pixel distances d_a , d_b and d_d from the center point C . Assuming that the image is rectilinear, such a pixel distance can be stated as a separation angle θ_i as follows:

$$\theta_i = \delta * \frac{d_i}{w} = \delta * \frac{|P_i - C|}{w} \quad (18)$$

Similar to equation 15, δ corresponds to the image's field of view angle and w to the diameter in pixels. As stated in equation 17, a cosine of an angle can be represented as a scalar product between two vectors. Furthermore, the scalar product between each position unit vector v_i and the view vector, hereby defined as c , should yield the cosine of the corresponding angular separation as stated below:

$$v_i \cdot c = \cos(\theta_i) \quad (19)$$

Using this correlation, we can formulate a system of linear equations with the position vectors v_a , v_b and v_d , such that its solution equals the view vector c , stated below in matrix multiplication notation:

$$\begin{bmatrix} v_{a_x} & v_{a_y} & v_{a_z} \\ v_{b_x} & v_{b_y} & v_{b_z} \\ v_{d_x} & v_{d_y} & v_{d_z} \end{bmatrix} * \begin{bmatrix} c_x \\ c_y \\ c_z \end{bmatrix} = \begin{bmatrix} \cos(\theta_a) \\ \cos(\theta_b) \\ \cos(\theta_d) \end{bmatrix} \quad (20)$$

Solving this equation for c yields the three values for the view vector, which should also be of uniform length 1. The view vector does not necessarily equate to the spacecraft's attitude, but may be used to determine it, which will be elaborated in sections 4.3 and 4.4.

4.2 Matching observed stars

The star matching strategy of our choice is the brightness independent 4-star matching algorithm by Ying, Fei and Zheng which we are going to expand upon and adapt in several places to improve performance. The main principle for identification of stars of this algorithm is basically that each two stars are separated by a fixed angle and sets of multiple stars can be characterized by their unique angular separations. The size of the set of stars to be identified has been fixed to four in this algorithm. The reason for that is that a quadruple of four stars can be characterized by six angular separations. While the number of stars is hereby quite small, the pattern of stars characterized by six separations may be very likely unique depending on the size of the underlying star catalog and some other parameters.

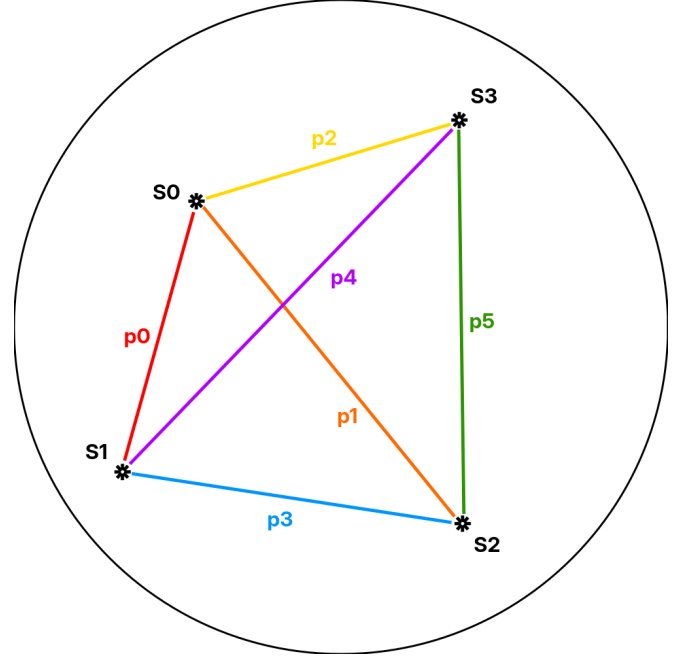


Figure 14: Four stars can be characterized by six angular separations.

As illustrated above in Figure 14, this algorithm tries to identify four stars by measuring and comparing their angular separations. The chosen four stars should be labeled S_0 , S_1 , S_2 and S_3 and their angular separation distances as p_0 , p_1 , p_2 , p_3 , p_4 and p_5 whereby p_0 is the separation between S_0 and S_1 , p_1 between S_0 and S_2 , p_2 between S_0 and S_3 , p_3 between S_1 and S_2 , p_4 between S_1 and S_3 and p_5 between S_2 and S_3 . [14]

Furthermore, the image frame used to pick stars from should be circular covering a field of view δ with a pixel width w . The circular shape is used to make the image rotation invariant such that only certain star pairs can fit within an image no matter its orientation. Moreover, Ying et al. define the data structure of a star pair. A star pair is a 3-tuple (i, j, θ) where i and j are identifier numbers referencing two stars and θ being the angular separation between these two stars [14]. Also the order of i and j does not matter so that $(i, j, \theta) = (j, i, \theta)$ can be considered valid.

For executing the algorithm, three data structures S_D , V_L and N_D have to be defined and computed beforehand, whereas the first two of them are requirements as defined by Ying et al. and the third one being an addition we are defining to improve performance.

1. First of all, the star dictionary S_D as $i \Rightarrow v$ needs to be set up. The values of this dictionary consist of a unit vector v , which can be derived from the right ascension α and declination ρ values of the BSC5P - Bright Star Catalog using equation 16. The keys of this dictionary i represent unique identifiers of the stars which will be used in following structures. [14]

2. Next up, all viable star pairs should be calculated and

stored in a list V_L . For this step all possible star pairs need to be calculated, which amount to a total of $\frac{n(n-1)}{2}$ for n stars. Conveniently, unit vectors and their corresponding identifiers have been determined in the previous data structure S_D . A star pair is to be considered viable if both of its stars can fit within the field of view δ of the image's frame. The angular separation θ thus must satisfy the requirement $\theta_{min} \leq \theta \leq \delta$, whereat θ_{min} can be considered as the minimum angle so that two stars can be told apart, which usually is just the angular diameter of one pixel.

Once all viable pairs have been determined, the list V_L needs to be sorted by angular separation of its star pairs in ascending order.[14]

3. Ultimately, we are going to define the neighbor dictionary N_D as $i \Rightarrow \{j, k, l, \dots\}$. This dictionary contains sets of star identifiers which correspond to a star identifier i . From this it is easily possible to determine which neighboring stars a star i possesses. Hereby, stars i and j are neighbors if there is a star pair in V_L which contains both identifiers.

This dictionary can be easily determined by iterating over V_L once.

Now that the preliminary data structures have been established, we can continue with the actual matching process. Ying et al. introduce a six step approach, whereof the first five steps are of relevance. Additionally the fifth step is insufficient, so we are going to expand upon that approach by adding some extra steps and modifications.

1. Four stars should be selected from the image and labeled as S_0, S_1, S_2 and S_3 . Ying et al. also add that lens distortion might be an issue and because of that the chosen stars should be located close to the center of the frame.[14]

However if the field of view δ is sufficiently small, lens distortion is of no issue. Although this algorithm is supposed to be brightness independent, we propose the modification that the brightest stars (within an image identifiable as largest dots) should be chosen. This is because they are most likely included within the star dictionary S_D and not some other object not included.

2. The six angular separations p_0, p_1, p_2, p_3, p_4 and p_5 between the four stars should be calculated [14].

3. As the viable star pair list V_L has been ordered by angular separation, candidate pairs may be determined. For each of the angular separations $p_{0\dots5}$, a candidate sublist l_i can be established, such that these sublists contain only star pairs (i, j, θ) with the condition $p_i - \epsilon \leq \theta \leq p_i + \epsilon$, whereupon ϵ is a small tolerance angle. [14]

4. A $n \times 6$ matrix M should be defined, whereat n equates to the number of stars within the star dictionary S_D . Hereby each of the n rows corresponds to a star from the dictionary and the columns correspond to the angular separations $p_{0\dots5}$. Each entry should be populated by either a 0 or a 1. If a star identifier $k = 1 \dots n - 1$ of

all n stars is contained within one of the star pairs of the candidate sublists l_i of the angular separations $p_{0\dots5}$, this entry should be set to 1. Otherwise it should be 0. [14]

5. Now, to match stars S_0, S_1, S_2 and S_3 with actual stars identifiers of S_D , Ying et al. propose an approach that may work but is certainly incomplete. If there is exactly one row in M that has the value [111000], the star identifier of S_0 has been identified. Equivalently for S_1 that would be [100110], for S_2 [010101] and for S_3 [001011]. If an entry is 1, it means that a star of index k can be found within the candidate star pair list l_i of $p_{0\dots5}$. [14]

This approach presents just a necessary criterion but not a sufficient one. A star identifier may occur within these candidate arrays, but it may not be an actual neighbor of the other three stars of the supposed four star quadruple. Furthermore, the star might also appear in other candidate lists l_i where it is not supposed to appear, e.g. [111001] with another occurrence in l_5 for the p_5 candidates might be just as well a valid candidate for S_0 .

As such, we are going to modify this step that no definite identifications for $S_{0\dots3}$ should be determined, rather candidate identifiers which will be stored in candidate arrays $C_{0\dots3}$ corresponding to $S_{0\dots3}$.

Now if there is one row in M that has the value [111***], the corresponding row index is a candidate for S_0 , thus gets added to C_0 . Hereby * may be an arbitrary value. Equivalently for S_1 that would be [1**11*], for S_2 [*1*1*1] and for S_3 [**1*11].

6. The candidate arrays $C_{0\dots3}$ corresponding to $S_{0\dots3}$ may now be reduced which yields either a fitting quadruple of four identified stars or nothing, if no identification is possible. For this, the algorithm will iterate multiple times over every candidate identifier k in every candidate array $C_{0\dots3}$.

An identifier k in array C_i must possess at least one neighboring identifier q or more in every other array $C_{j \neq i}$. This step requires the previously defined neighbors dictionary N_D . Furthermore, the angular separation θ between k and at least one of these neighbors q must match the corresponding measured angular separation p within defined tolerance limits $p - \epsilon \leq \theta \leq p + \epsilon$.

If these conditions are met, k may remain in C_i for now, otherwise k is removed immediately. The reduction process repeats until the summed up length of $C_{0\dots3}$ does not change anymore or is smaller than four.

In the latter case, no identifiers for a quadruple could be matched, thus the identification has failed. Ideally one star identifier k per candidate array C_i remains, which means that the procedure was successful. Occasionally, two or more stars may remain in a candidate array. This is the case if two stars of a binary system are both listed within the star catalog or the apparent separation between two unrelated stars is small enough.

If multiple stars remain in every candidate array, it is not possible to determine a definite match.

4.3 Simple attitude determination

Once stars have been identified within an image and a view vector has been established, the attitude of the spacecraft may be determined. The most basic approach is to simply turn the spacecraft so that it faces the opposite direction of the sun or let the spacecraft face the sun and have a star camera mounted so that it faces backward. Determining the view vector of that image directly corresponds to the attitude of the spacecraft, as illustrated in Figure 15 below.

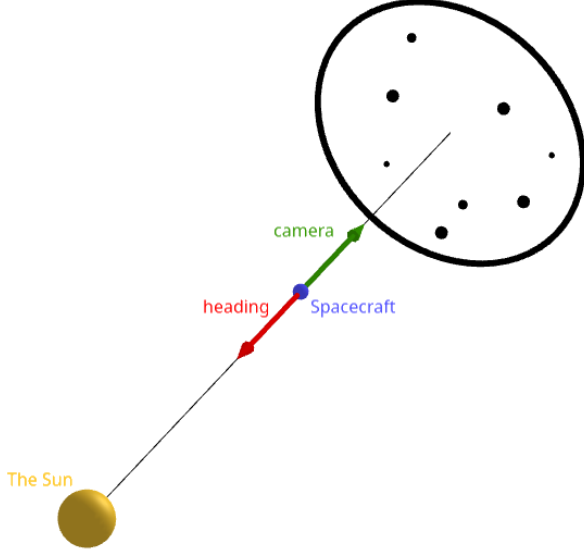


Figure 15: View vector and heading vector aligning in such a way that the view vector equates attitude.

As attitude is defined as right ascension α and declination ρ , these values have to be extracted from the view vector c . The Fundamental Technologies Voyager LECP Pages website provides us with equations for exactly this task:

$$\begin{aligned}\rho &= \arcsin(c_z) \\ \alpha &= \arctan\left(\frac{c_y}{c_x}\right)\end{aligned}\quad (21)$$

This approach works if star matching is successful and three known stars can be located right behind the spacecraft. If that is not the case, then attitude determination is not possible using this approach, which is quite disadvantageous. Another more advanced approach will be elaborated in section 4.4.

4.4 Optimization strategies

The process of attitude determination entails the steps of running a star matching algorithm and performing an attitude determination procedure, whereat view vector determination is part of the latter. These are also the steps where optimization approaches may be applied to. As such, we are going to elaborate these approaches separately.

4.4.1 Matching algorithm optimization

The presented brightness independent 4-star matching algorithm, enhancements applied, relies on the preemptively computed data structures, the star dictionary S_D , the sorted viable star pair list V_L and the neighbors dictionary N_D . Specifically relevant in terms of performance at runtime are the viable star pair list V_L and the neighbor dictionary N_D as the algorithm accesses these data structures.

The larger the size of V_L , the more candidate star pairs would end up in the candidate lists l_i in step 3. This could lead to more row matches in step 5, thus more candidate stars in the candidate star arrays $C_{0...3}$. In step 6 this could lead to multiple remaining stars per candidate array C_i so that it would be impossible to determine a match. Therefore it seems advantageous to keep the size of the viable star pair list V_L sufficiently small.

The number of star pairs within V_L is directly influenced by the number of stars within the catalog dictionary S_D and the camera's field of view δ . As for the catalog dictionary S_D , it may be advantageous to limit the number of guide stars taken from the underlying catalog. The BSC5P - Bright Star Catalog contains more than 9000 entries which include data on the apparent brightness of a star, characterized by the apparent magnitude [13]. As we can control camera sensitivity so that we can limit the brightness of stars, the camera can detect, we can also limit the stars in our catalog dictionary S_D . Stars invisible to the camera do not need to be part of the dictionary, as they cannot be detected. In addition to that, they also should not be part of the dictionary because it could lead to misidentification as the matching algorithm could now accidentally identify stars which are not even visible to the camera. Generally speaking, the set of stars detectable by the camera should be mostly identical to the set of stars in the catalog dictionary S_D . This minimizes the possibility of mismatches and the possibility of observed stars not being assignable to catalog stars. This set issue is illustrated below in Figure 16.

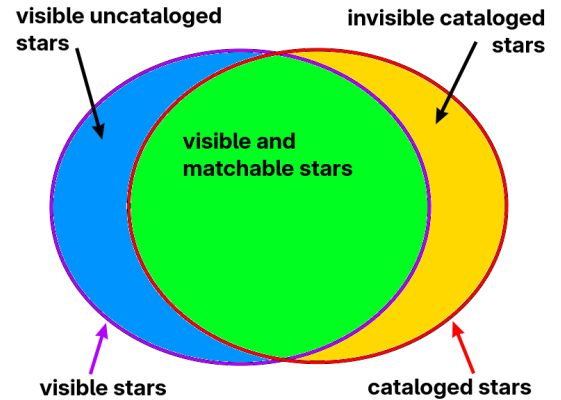


Figure 16: The overlap of the visible stars and the cataloged stars can be assigned by the matching algorithm.

Limiting guide stars in S_D subjected to apparent magnitude greatly affects the number of viable star pairs. To determine how the number of stars changes, we can look at the number of stars brighter than a certain magnitude by filtering entries of the BSC5P - Bright Star Catalog.

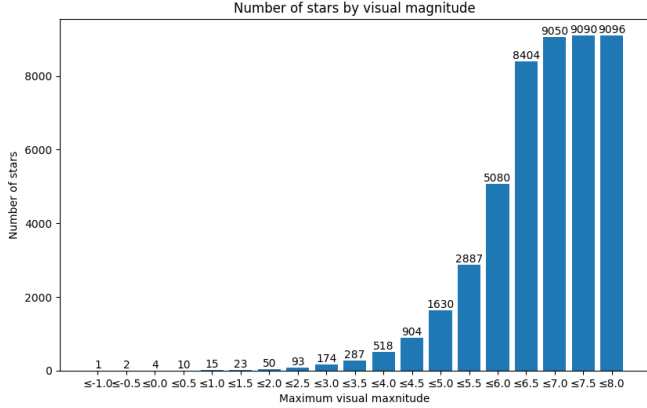


Figure 17: The number of stars brighter than a certain apparent magnitude.

As illustrated above in Figure 17, the number of visible stars approximately increases exponentially up to a magnitude of 6.5 as this value is being increased. Actually the BSC5P catalog should only contain objects up to that value [13], however this might not be a hard limit as there seem to be some dimmer objects included.

The key takeaway of this plot is that increasing camera sensitivity to detect stars of dimmer magnitude increases the number of visible stars or number of stars in the catalog dictionary S_D exponentially.

Another tuneable parameter is the camera's field of view. The angular diameter δ is directly responsible for the number of viable star pairs within V_L as a star pair can only be considered viable if both stars fit inside a frame. Now that the size of V_L directly depends on δ and indirectly on the magnitude limit responsible for the size of S_D , we can calculate the size of V_L exemplarily for different parameter values.

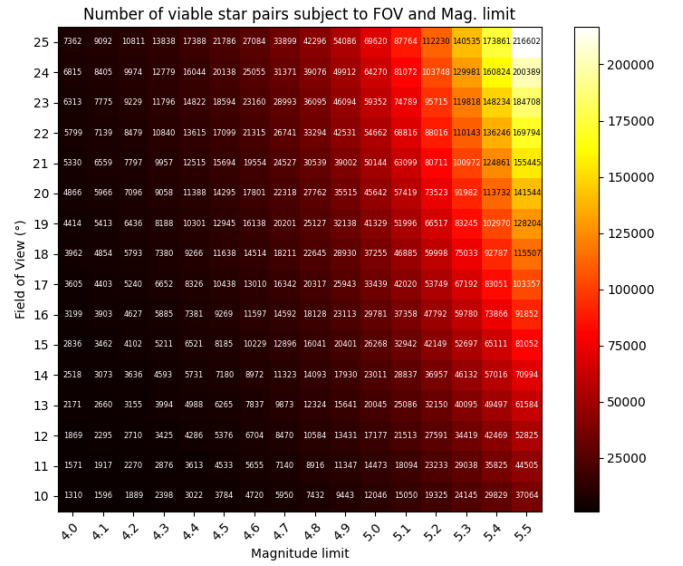


Figure 18: Number of viable star pairs in V_L subject to field of view and magnitude limit.

The above plot in Figure 18 shows the number of star pairs within V_L depending on the field of view and the magnitude limit. The upper limit of 25° is based on the field of view value of 24.5° chosen by Ying et al. during evaluation of their algorithm [14], whereas the magnitude limit range from 4.0 to 5.5 offers a reasonable amount of guide stars. Evidently, the number of viable pairs increases in an exponential fashion similar to Figure 17 for any fixed field of view value as the magnitude limit increases. For any fixed magnitude limit, the number of viable pairs seems to increase in a squared fashion as the field of view value increases.

Now that the size change of V_L follows a predictable behavior, we can go on to choose more concrete values for δ and the magnitude limit. Certainly though, there is no perfect solution, which can be determined simply from this plot, as some more aspects have to be considered.

One requirement might be that the entirety of the surrounding sky needs to be viable for attitude determination, which is exactly the main requirement of the simple attitude determination procedure, as elaborated in section 4.3.

To be able to determine attitude, no matter the position relative to the sun, at least four stars must be visible in every possible camera frame for them to be identifiable by our algorithm. This demand can be tested by selecting arbitrary values for the field of view and the magnitude limit and counting the stars for each direction the camera may be pointed at. Hereby every direction can be covered by iterating over right ascension from 0° to 359° and declination from 90° to -90° , each with a step size of 1° . This yields a star density map for the entire celestial sphere, whereof spots with insufficient star density may become visible.

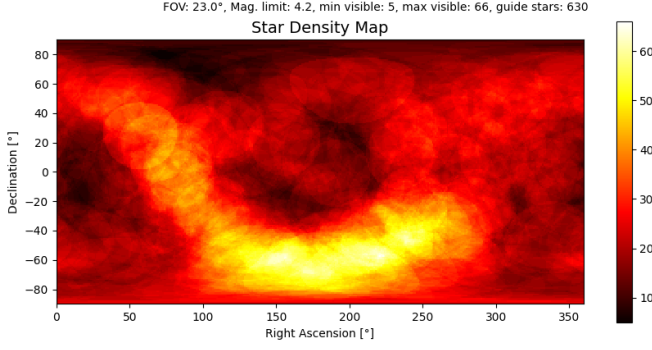


Figure 19: Star density map for a field of view of 23° and a magnitude limit of 4.2.

As shown above in Figure 19, this procedure has been conducted exemplary for a field of view of 23° and a magnitude limit of 4.2. Guide stars are unevenly distributed in the celestial sphere so that the minimum number of visible stars is only 5, while the maximum number is 66. But because at least five stars are visible in every direction, those camera properties satisfy the requirement of the simple attitude determination procedure, so that there are at least four stars in every possible camera frame. Looking at Figure 18, the magnitude limit is located near the lower bound, while the field of view is near the upper bound. Furthermore, we can look up the number of viable star pairs, which amount to only 9229.

This approach makes it seem advantageous to generally choose a larger field of view value, instead of increasing camera sensitivity to capture dimmer stars. Generally speaking this is correct. However, there are also some disadvantages associated with a larger field of view. One disadvantage is the aspect that at some point lens distortion becomes an issue. Although this distortion can be corrected, as shown in section 2.2, this adds a layer of complexity, which can simply be avoided by choosing a smaller field of view such that distortion is negligible. Another disadvantage is the aspect of decreasing precision if the pixel width of an image stays constant while the field of view is being increased. As the triangulation of a view vector relies on extracted separations from an image, as elaborated in 4.1, a worse per pixel resolution leads to an imprecise view vector.

Eventually there is no perfect pair of values for field of view and magnitude limit. When defining these properties for a camera system with our star matching algorithm, one may be determined by additional constraints, while the other one may be determined using the presented approach.

4.4.2 Advanced attitude determination

The previously in section 4.3 defined simple attitude determination procedure relies on the principle, that there is always an identifiable four star group in an image frame facing away from the sun. However, if the set of visible stars does not perfectly overlap with the set of stars in the

catalog dictionary S_D , there can be occurrences where visible stars can not be assigned. Furthermore, there are multiple objects in space that could be misidentified as stars, such as planets, asteroids, globular clusters or other galaxies. Essentially, attitude determination can not be guaranteed on the basis of a single image frame fixed to a certain direction.

To solve this issue, we introduce an advanced attitude determination procedure where the spacecraft starts off facing the sun. Next up, it rotates counterclockwise around its perpendicular axis by a known angle ω . From this angle attitude determination is attempted. If unsuccessful, ω has to be increased and the procedure is repeated until attitude has been successfully determined. A detailed model is showcased below in Figure 20

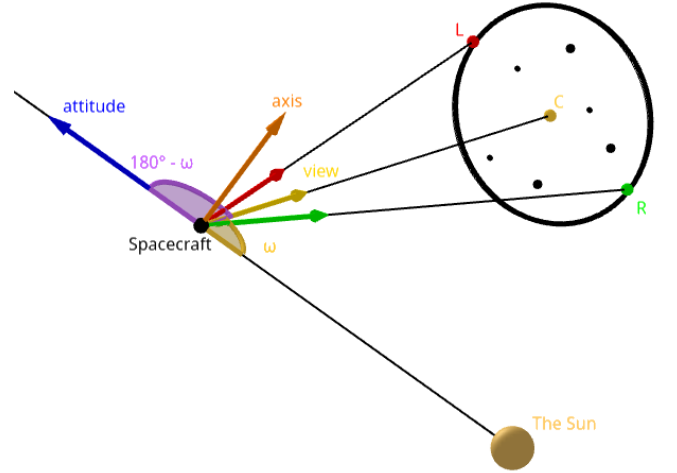


Figure 20: Attitude determination by rotating the spacecraft around a known angle and taking multiple images.

The spacecraft has been turned around an angle ω . Under the assumption that star matching has been successful, the view vector c can be determined. Equivalently, the right view r and left view l vector can be determined. In section 4.1 the view vector is triangulated from a point in the image center. The left and right view vectors l and r can be similarly triangulated by placing these points at the right and left edge of an image instead of the center. Afterwards, an axis vector a can be determined by calculating the cross product between r and l as follows:

$$a = \frac{1}{|r \times l|} * (r \times l) \quad (22)$$

Hereby we additionally norm the axis vector so that it is a unit vector. Next up, the paper by Cheng and Gupta provides the Euler-Rodrigues rotation formula. This formula rotates a vector u around a unit axis vector k by an angle ϕ as follows:

$$u' = u * \cos(\phi) + (k \times u) * \sin(\phi) + k * (k \cdot u) * (1 - \cos(\phi)) \quad (23)$$

[16]

Hereby we insert c as u , a as k and $180^\circ - \omega$ as ϕ which

yields the rotated view vector c' . This rotated view vector c' is similar to the view vector of the simple attitude determination as it faces away from the sun. However, the imaging direction from which it has been determined is a different one. Equivalently, the formulas 21 can be applied to c' to gather the values for right ascension α and declination ρ .

This approach eliminates the disadvantage of the simple attitude determination procedure as it does not rely on a single imaging direction, thus being unconcerned about disruptive factors.

4.4.3 Utilizing all stars within an image

As illustrated in Figure 19, areas with very few stars in the celestial sphere are exceptional cases. Typically, there are more than four stars in an image frame. By default the matching algorithm in section 4.2 selects only four stars in an image. If matching these four stars is unsuccessful, it would be reasonable to select a different quadruple of four stars in the image frame if more than four stars are available. In fact, if a frame entails n stars it is possible to form m quadruples, whereas the number of m can be calculated as the binomial coefficient as follows:

$$m = \binom{n}{4} = \frac{n!}{4!(n-4)!} \quad (24)$$

Hereby k equals 4, as there must always be four stars in a quadruple. This approach is particularly useful as an addition to the simple attitude determination procedure as there are now more possibilities to determine attitude from a single direction.

5 Conclusion

Full self localization in interplanetary space can be achieved by using purely image based approaches. Three major steps have been devised, which when performed in sequence, yield the position of the spacecraft in the defined spherical coordinate system.

1. Localization of the sun: The sun can be localized by performing an imaging survey, which captures the entire surrounding sky in only six images. From these images the exact direction of the sun can be determined. By performing a sequence of rotations, the camera can be pointed in the direction of the sun.

As this imaging survey uses a wide field camera, radial lens distortion is an issue. This issue can be corrected with a polynomial corrective function. The coefficients of this function are determined with a calibration procedure which fits observed points to supposed rectilinear points by minimizing the error of an objective function.

2. Determining distance to the sun Once the sun has been localized, the distance to the sun can be determined by making use of a simple geometrical model by comparing apparent angular diameter to the camera's

field of view and using some trigonometry.

Camera properties have been loosely based on parameters of the New Horizons spacecraft's camera LORRI. The approach yields fairly accurate results over a range of various distances. Above distances of about 100 astronomical units, estimations become inaccurate.

3. Determining attitude to the background stars:

Determining attitude to the background stars itself is composed of multiple steps. At first, four observed stars within an image need to be matched to stars in a catalog using a matching algorithm based on comparing angular distances between stars. From these matched stars a view vector can be triangulated. By using a simple or advanced attitude determination procedure, right ascension and declination can be determined from the view vector.

Furthermore, the matching algorithm's performance can be tuned by selecting specific camera parameters, namely the field of view and camera sensitivity to detect stars up to a certain magnitude limit.

References

- [1] "Distance measurement" *How NASA Calculates the Distance, Velocity and Position of the Voyager Spacecrafts*, Karthikeyan KC, 2015, <https://geekswipe.net/technology/aerospace/how-nasa-calculates-the-distance-velocity-and-position-of-the-voyager-spacecrafts/>. Accessed 03.10.2025
- [2] Star Tracker, NASA, 2019, <https://exoplanets.nasa.gov/star-tracker/>. Accessed 03.10.2025
- [3] "About" *Space Engine*, Cosmographic Software LLC, 2025, <https://spaceengine.org/manual/making-addons/scenario-scripts/>. Accessed 06.10.2025
- [4] "Equatorial coordinate system", *COSMOS - The SAO Encyclopedia of Astronomy*, Swinburne University of Technology, <https://www.astronomy.swin.edu.au/cosmos/E/Equatorial+Coordinate+System>. Accessed 04.10.2025
- [5] "Skybox", Valve Developer Community, 2024, [https://developer.valvesoftware.com/wiki/Skybox_\(2D\)](https://developer.valvesoftware.com/wiki/Skybox_(2D)). Accessed 05.10.2025
- [6] B. Prescott and G. F. McLean. "Camera Calibration", Line-Based Correction of Radial Lens Distortion, 1996, pp. 2-3, <https://www.sciencedirect.com/science/article/abs/pii/S1077316996904074>. Accessed 11.10.2025
- [7] Prof. Dr. Markus Schneider. "Nonlinear Modeling, Model Selection", *Nonlinear Modeling and Generalization*, 5812 Machine Learning Lecture Notes, 03.2024, pp.29-32. Accessed 04.07.2024
- [8] Ignasi Ribas. "Introduction", The Sun and stars as the primary energy input in planetary atmospheres, 2010, p. 4, <https://doi.org/10.1017/S1743921309992298>. Accessed 15.10.2025
- [9] A.F. Cheng et al. "LORRI Requirements", Long-Range Reconnaissance Imager on New Horizons, 2007, p. 192, <https://link.springer.com/article/10.1007/s11214-007-9271-6>. Accessed 17.10.2025
- [10] The Editors of Encyclopaedia Britannica. "Astronomical unit", unit of measurement, <https://www.britannica.com/science/astronomical-unit>. Accessed 17.10.2025
- [11] NASA Science Editorial Team. "NASA's Parker Solar Probe Sails Through 25th Sun Flyby", Parker Solar Probe, 2025, <https://science.nasa.gov/blogs/parker-solar-probe/2025/09/18>. Accessed 18.10.2025
- [12] NASA. "Mission Status", Where are Voyager 1 and Voyager 2 Now?, 2025, <https://science.nasa.gov/mission/voyager/where-are-voyager-1-and-voyager-2-now>. Accessed 18.10.2025
- [13] Browse Software Development Team. BSC5P - Bright Star Catalog, 2024, <https://heasarc.gsfc.nasa.gov/w3browse/star-catalog/bsc5p.html>. Accessed 12.09.2025
- [14] Dong Ying, Xing Fei, You Zheng. Brightness Independent 4-Star Matching Algorithm for Lost-in-Space 3-Axis Attitude Acquisition, 2006, <https://www.sciencedirect.com/science/article/abs/pii/S1007021406702322>. Accessed 15.08.2025
- [15] "Compute the Right Ascension and Declination Angles", *Right Ascension and Declination Algorithm*, Fundamental Technologies Voyager LECP Pages, 2002, https://voyager-mac.umd.edu/docs/ftec_2006/SEDR/algorithm.html. Accessed 19.10.2025
- [16] Hui Cheng, K. C. Gupta. "Introduction", An Historical Note on Finite Rotations, 1989, p. 1, https://iel.ucdavis.edu/publication/1989/j_ASMEAM.pdf. Accessed 21.10.2025